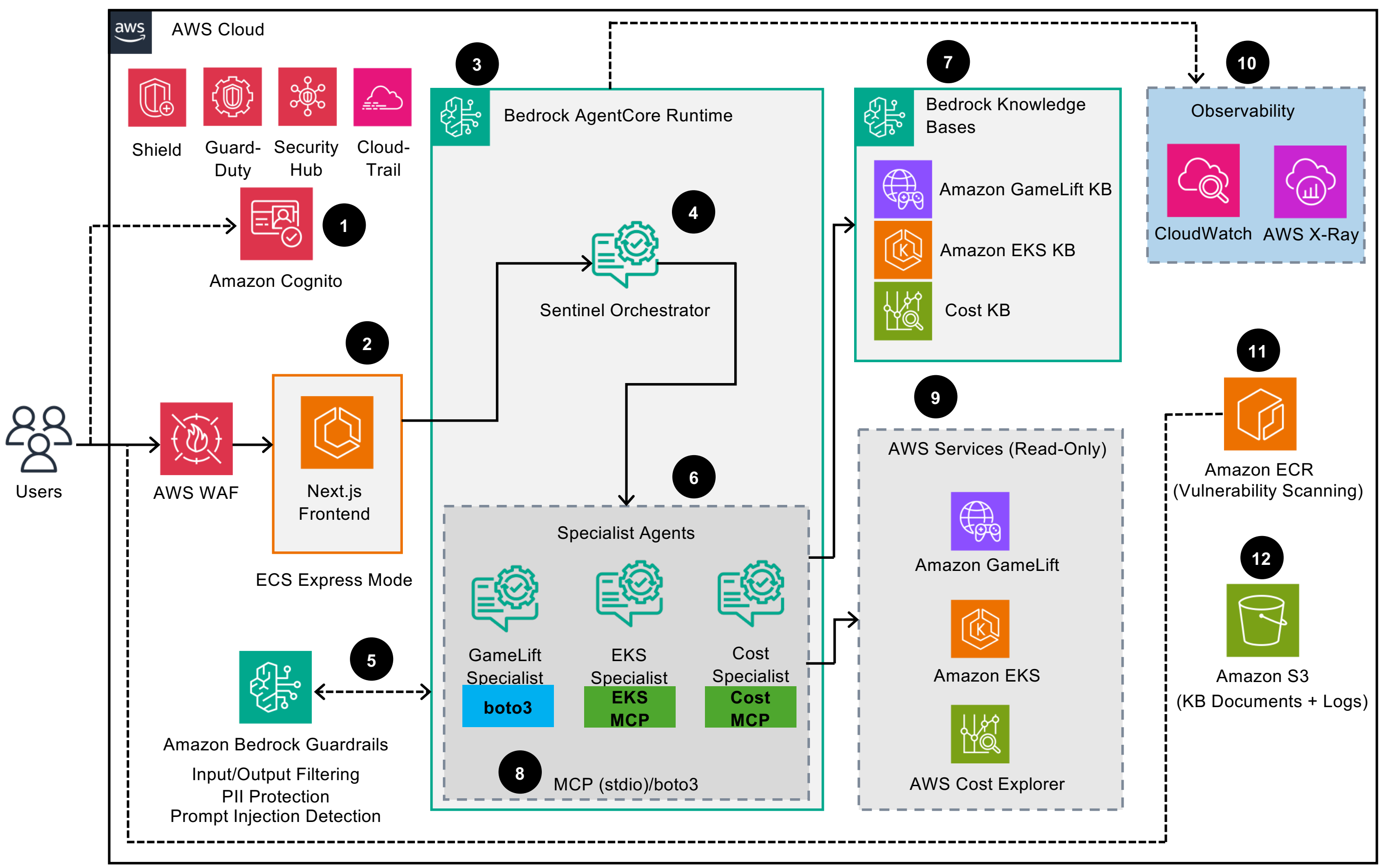
Project Sentinel - AI-Powered Game Server Management

This architecture diagram shows how to use AWS Bedrock AgentCore with specialized AI agents to provide conversational management of Amazon GameLift and Amazon EKS infrastructure.



Architecture Flow

1. Authenticate with **Amazon Cognito** User Pool. JWT tokens stored in HitpOnly cookies.
2. Send query through **Next.Js frontend** on **AWS App Runner**. CopilotKit provides the chat UI.
3. Invoke **Bedrock AgentCore Runtime** via AWS SDK with SigV4 authentication.
4. Route to **Sentinel Orchestrator** agent which classifies query intent.
5. **Bedrock Guardrails** filter input/output. Detects prompt injection, blocks Pil exposure.
6. Delegate to **Specialist Agent** (GameLift, EKS, or Cost) based on query
7. Query **Bedrock Knowledge Base** for domain context. RAG with Titan embeddings.
8. Invoke **MCP Servers** (EKS, Cost) or boto3 (GameLift) for AWS API integration.
9. Read-only API calls to **AWS Services**: GameLift fleets, EKS clusters, Cost analysis.
10. **CloudWatch logs** and **X-Ray** traces capture observability data.
11. Container images in **Amazon ECR** with vulnerability scanning enabled.
12. KB documents stored in **Amazon S3** with encryption and access logging.